Fiche récapitulative : Git et GitHub pour un projet web à deux

Cette fiche explique les outils à utiliser, le vocabulaire, et surtout le fonctionnement des **branches** pour travailler à deux sur un site web sans s'écraser mutuellement le travail.

1. Git et GitHub : quelle différence ?

- **Git** est un logiciel installé sur votre ordinateur. Il enregistre l'historique de votre projet (chaque modification, qui l'a faite, quand) et permet de revenir en arrière.
- **GitHub** est un site web qui héberge votre projet en ligne. Il sert de « dépôt central » que vous partagez, et ajoute des outils de collaboration (Pull Requests, revues de code, gestion des tâches).

Une image simple : Git est le **carnet de versions** sur votre poste, GitHub est le **classeur partagé** dans le cloud.

2. Les outils à installer

Outil	Rôle	Obligatoire ?
Git (git-scm.com)	Le logiciel de gestion de versions	Oui
Compte GitHub (github.com)	Héberger et partager le projet	Oui (un compte chacun)
Visual Studio Code	Éditeur de code, avec Git intégré	Recommandé
GitHub Desktop	Interface graphique pour Git, sans ligne de commande	Optionnel, utile pour débuter
Un terminal	Pour taper les commandes Git (celui de VS Code suffit)	Recommandé

Conseil : pour débuter, vous pouvez tout faire avec GitHub Desktop ou avec l'onglet « Source Control » de VS Code. Apprendre les commandes reste utile, car elles fonctionnent partout et aident à comprendre ce qui se passe.

3. Vocabulaire essentiel

Terme	Signification
Dépôt (repository, « repo »)	Le dossier du projet suivi par Git
Commit	Une « photo » enregistrée de votre projet à un instant donné, avec un message qui explique ce qui a changé
Branche (branch)	Une ligne de travail parallèle. Elle permet de développer une fonctionnalité sans toucher à la version principale
main	La branche principale, qui doit toujours contenir une version propre et fonctionnelle du site
Clone	Télécharger une copie d'un dépôt GitHub sur son ordinateur
Push	Envoyer ses commits locaux vers GitHub
Pull	Récupérer depuis GitHub les modifications des autres
Fetch	Regarder ce qui a changé sur GitHub sans l'appliquer
Merge (fusion)	Intégrer les modifications d'une branche dans une autre
Pull Request (PR)	Une demande, faite sur GitHub, pour fusionner votre branche dans `main`. Votre collègue peut relire et commenter avant
Conflit	Situation où deux personnes ont modifié les mêmes lignes d'un fichier : Git demande de choisir quelle version garder
Origin	Le nom habituel du dépôt distant sur GitHub
Staging (index)	La zone d'attente où l'on place les fichiers qu'on va inclure dans le prochain commit

4. Le principe de fonctionnement

Il existe trois « endroits » où vivent vos fichiers :

1. **Votre dossier de travail** : les fichiers que vous modifiez.
2. **Votre dépôt local** : l'historique enregistré par Git sur votre ordinateur.
3. **Le dépôt distant (GitHub)** : la copie partagée.

Le trajet d'une modification est toujours le même :

modifier des fichiers → `git add` (préparer) → `git commit` (enregistrer en local) → `git push` (envoyer sur GitHub)

Et pour récupérer le travail de l'autre : **`git pull`**.

5. Configuration initiale (à faire une seule fois, chacun de son côté)

Après avoir installé Git, ouvrez un terminal et tapez :

```
git config --global user.name "Votre Nom"
git config --global user.email "votre-email@exemple.com"
```

Utilisez la même adresse e-mail que celle de votre compte GitHub, pour que vos contributions soient bien associées à votre profil.

6. Mise en place du projet

Étape A : une seule personne crée le dépôt

1. Sur GitHub, cliquer sur **New repository**.
2. Donner un nom (par exemple `mon-site-web`), choisir **Private** si le projet ne doit pas être public.
3. Cocher **Add a README file** (ça crée directement la branche `main`).
4. Dans **Settings → Collaborators**, inviter votre collègue avec son nom d'utilisateur GitHub. Il doit accepter l'invitation reçue par e-mail.

Étape B : chacun clone le dépôt

Sur GitHub, cliquer sur le bouton vert **Code** et copier l'adresse HTTPS, puis :

```
git clone https://github.com/votre-compte/mon-site-web.git
cd mon-site-web
```

Git crée un dossier avec tout le projet. Vous travaillez désormais dedans.

Étape C : ajouter un fichier `.gitignore`

Ce fichier liste ce que Git doit **ignorer** (fichiers temporaires, dossiers générés, mots de passe). Exemple de contenu pour un projet web :

```
node_modules/
.env
.DS_Store
Thumbs.db
dist/
build/
```

Ne commitez jamais de mots de passe, clés d'API ou fichiers `.env`.

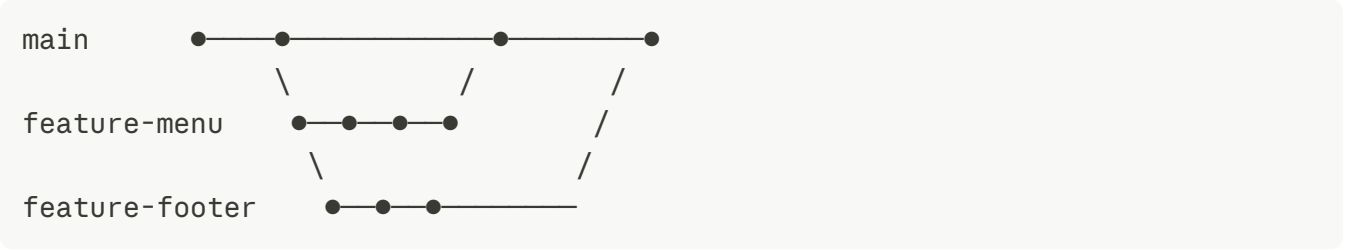
7. Le système de branches

Pourquoi des branches ?

Si vous travaillez tous les deux directement sur `main`, vous risquez de vous marcher dessus, et une modification à moitié finie peut casser le site. Avec les branches :

- `main` reste **stable** : c'est la version qui fonctionne.
- Chacun travaille sur **sa propre branche** pour une tâche précise.
- Quand la tâche est finie, on la **fusionne** dans `main` via une Pull Request.

Schéma simplifié



Chaque point est un commit. Les branches `feature-...` partent de `main`, évoluent séparément, puis reviennent dans `main`.

Convention de nommage des branches

Choisissez des noms courts, en minuscules, avec des tirets, qui décrivent la tâche :

- `feature/page-contact`
- `feature/menu-navigation`
- `fix/bug-affichage-mobile`
- `style/couleurs-header`

Préfixes courants : `feature/` (nouvelle fonctionnalité), `fix/` (correction), `style/` (design), `docs/` (documentation).

Commandes pour gérer les branches

Action	Commande
Voir les branches locales et la branche actuelle	`git branch`
Voir aussi les branches distantes	`git branch -a`
Créer une branche et s'y placer	`git switch -c feature/page-contact`
Changer de branche	`git switch nom-de-la-branche`
Envoyer une nouvelle branche sur GitHub	`git push -u origin feature/page-contact`
Supprimer une branche locale (après fusion)	`git branch -d feature/page-contact`
Supprimer une branche sur GitHub	`git push origin --delete feature/page-contact`

(Sur d'anciens tutoriels, vous verrez `git checkout` à la place de `git switch` : c'est équivalent pour changer de branche.)

8. Le cycle de travail complet, pas à pas

Voici la routine à suivre à chaque nouvelle tâche.

1. Se mettre à jour sur main

```
git switch main
git pull
```

2. Créer sa branche de travail

```
git switch -c feature/page-contact
```

3. Travailler, puis enregistrer régulièrement

```
git status                      # voir ce qui a changé
git add .                       # préparer tous les fichiers modifiés
git commit -m "Ajoute le formulaire de contact"
```

Faites des **petits commits fréquents**, chacun avec un message clair qui décrit ce qui a été fait.

4. Envoyer sa branche sur GitHub

La première fois :

```
git push -u origin feature/page-contact
```

Les fois suivantes, un simple `git push` suffit.

5. Ouvrir une Pull Request

1. Sur GitHub, un bandeau jaune propose **Compare & pull request** : cliquer dessus (sinon, onglet **Pull requests → New pull request**).
2. Vérifier que la base est `main` et que la branche à fusionner est la vôtre.
3. Écrire un titre et une courte description de ce qui a été fait.
4. Demander à votre collègue d'être **Reviewer**.
5. Cliquer sur **Create pull request**.

6. Relecture et fusion

Votre collègue lit les modifications (onglet **Files changed**), peut commenter, puis clique sur **Merge pull request**. La branche est alors intégrée dans `main`. GitHub propose ensuite de supprimer la branche : acceptez.

7. Récupérer la version à jour et recommencer

```
git switch main
git pull
```

Puis retour à l'étape 2 pour la tâche suivante.

9. Récupérer le travail de l'autre pendant qu'on travaille

Si votre collègue a fusionné quelque chose dans `main` pendant que vous travaillez sur votre branche, vous pouvez intégrer ces changements dans votre branche :

```
git switch main
git pull
git switch feature/page-contact
git merge main
```

Faites-le régulièrement, surtout avant d'ouvrir votre Pull Request : cela limite les conflits.

10. Gérer un conflit

Un conflit arrive quand vous avez tous les deux modifié les mêmes lignes. Git ne devine pas, il vous demande de trancher. Dans le fichier concerné, vous verrez :

```
<<<<<<< HEAD
Votre version
=======
La version de l'autre
>>>>>>> main
```

Pour résoudre :

1. Ouvrir le fichier (VS Code propose des boutons « Accept Current / Incoming / Both »).
2. Choisir ou combiner les deux versions, puis **supprimer les marqueurs** `<<<<<<<`, `=======`, `>>>>>>>`.
3. Enregistrer, puis :

```
git add .
git commit -m "Résout le conflit sur la page d'accueil"
```

Astuce : le meilleur moyen d'éviter les conflits est de se répartir les fichiers (par exemple l'un sur le header, l'autre sur le footer) et de communiquer sur qui touche à quoi.

11. Règles de bonne collaboration

- **Ne jamais travailler directement sur `main`.** Toujours passer par une branche et une Pull Request.
- **Un sujet = une branche.** Évitez les branches fourre-tout.
- **Faire un `git pull` avant de commencer** à travailler, chaque jour.
- **Commiter souvent**, avec des messages explicites (« Ajoute le menu responsive » plutôt que « modifs »).
- **Relire les Pull Requests de l'autre** : c'est aussi un bon moyen d'apprendre.
- **Se parler** : annoncer sur quoi on travaille pour éviter de modifier les mêmes fichiers.

- **Protéger `main`** (optionnel) : dans **Settings → Branches → Add rule**, cocher « Require a pull request before merging » pour interdire les envois directs sur `main`.

12. Aide-mémoire des commandes

Besoin	Commande
Cloner un dépôt	`git clone <url>`
Voir l'état du projet	`git status`
Voir l'historique des commits	`git log --oneline`
Préparer tous les fichiers	`git add .`
Préparer un fichier précis	`git add nom-du-fichier`
Enregistrer un commit	`git commit -m "message"`
Envoyer sur GitHub	`git push`
Récupérer depuis GitHub	`git pull`
Voir les différences non enregistrées	`git diff`
Créer et changer de branche	`git switch -c nom`
Changer de branche	`git switch nom`
Fusionner une branche dans la branche actuelle	`git merge nom`
Annuler les modifications d'un fichier non commité	`git restore nom-du-fichier`
Retirer un fichier de la préparation (avant commit)	`git restore --staged nom-du-fichier`

13. Problèmes courants

Message ou situation	Cause probable et solution
`rejected ... non-fast-forward` au push	Votre collègue a envoyé des modifications avant vous. Faites `git pull`, résolvez les éventuels conflits, puis `git push`

Message ou situation	Cause probable et solution
`You have unstaged changes` / impossible de changer de branche	Des modifications ne sont pas enregistrées. Commitez-les, ou mettez-les de côté avec `git stash` (puis `git stash pop` pour les récupérer)
Je suis sur la mauvaise branche	Vérifiez avec `git branch` avant de commiter. Si vous avez déjà commité sur `main` par erreur, demandez de l'aide avant de forcer quoi que ce soit
Demande d'identifiants au push	Connectez-vous via le navigateur quand Git le propose, ou utilisez GitHub Desktop / VS Code qui gèrent l'authentification
J'ai commité un mot de passe	Changez immédiatement ce mot de passe, puis cherchez comment nettoyer l'historique : le supprimer dans un nouveau commit ne suffit pas

Réflexe de sécurité : `git status` est sans danger et vous dit toujours où vous en êtes. Utilisez-le sans modération. Évitez `git push --force` et `git reset --hard` tant que vous n'êtes pas à l'aise : ils peuvent détruire du travail.

14. Pour aller plus loin

- **GitHub Issues** : une liste de tâches et de bugs, que l'on peut assigner à l'un ou à l'autre.
- **GitHub Projects** : un tableau type Kanban (À faire / En cours / Terminé).
- **GitHub Pages** : héberger gratuitement un site statique directement depuis le dépôt.
- **Documentation officielle** : docs.github.com et git-scm.com/book (le livre « Pro Git » existe en français).

15. Résumé en 6 lignes

1. `git switch main` puis `git pull` : se mettre à jour.
2. `git switch -c feature/ma-tache` : créer sa branche.
3. Travailler, puis `git add .` et `git commit -m "message"` : enregistrer.
4. `git push -u origin feature/ma-tache` : envoyer sur GitHub.
5. Ouvrir une **Pull Request**, faire relire par son collègue, **fusionner**.
6. Retour sur `main`, `git pull`, et on recommence.